Applied Data Science Department

Specializing Large Language Models using Chain of Thought Reasoning

Project Advisor: Prof. Simon Shim

Chavidi, Anvesh Kumar,
Gupta, Eshita
Potula, Sandeep Reddy
Thakur, Neha
Varshney, Saumya

Introduction

Large Language Models (LLMs) excel at factual and comprehension tasks but struggle with complex reasoning. This project fine-tunes open-source LLMs using Chain-of-Thought (CoT) prompting combined with techniques like GRPO, DPO, Chain of Draft, Self-Consistency, and Tree-of-thought to improve reasoning transparency and accuracy. The system is designed as an intelligent tutoring assistant for math and legal domains.

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ✗

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

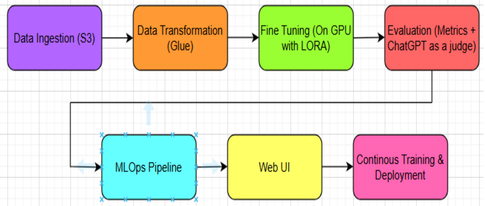
Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

Methodology

Our methodology fine-tuned four open-source LLMs using LoRA adapters. Implemented multiple reasoning strategies Chain-of-Thought, Self-Consistency, GRPO, and Tree-of-Thought were applied to improve multi-step logical reasoning. A cloud-based pipeline (AWS S3, Glue, Redshift, RDS) handled data ingestion and transformation, and models were benchmarked on math test problems.



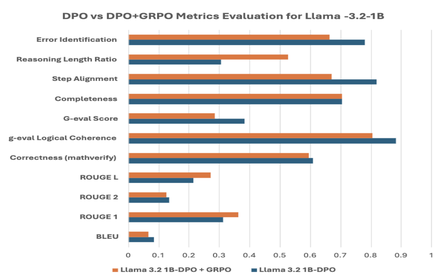
- **Models**: LLaMA3, Gemma3, Phi3, DeepseekR1
- **Fine-Tuning**: LoRA on GPU and Unsloth
- **Reasoning Strategies**: Chain of Draft, GRPO, DPO, Tree-of-Thought, Self-Consistency
- **Data Pipeline**: AWS (S3, Glue, Redshift, RDS)
- **Datasets**: MATH (Hugging face, Educational institutions, Educational wikis)

Analysis and Results

To evaluate the effectiveness of our reasoning strategies, conducted both quantitative and qualitative analyses using multiple open metrics and model behaviors. The following results highlight the performance of our fine-tuned LLMs across different techniques and datasets.

Comparison of techniques across different models

The Llama 3.2 1B-DPO model achieves the highest G-eval score of 0.38378, making it the best-performing model among the models that we tested. Key insights include the impressive performance of DPO-based fine tuning technique, particularly with the Llama 3.2 1B model, while combining GRPO with DPO or adding COT/COD generally yields lower scores, except for a slight improvement with COT in Llama 3.2 3B-GRPO. DeepSeek models perform poorly, with R1-SC scoring the lowest at 0.04, indicating significant variability in model effectiveness across different techniques.



Fine-Tuned Model Self-Correction Example

A standout example of reasoning improvement is illustrated here, where the fine-tuned LLaMA 3.2 model identifies a contradiction in its initial solution path and revises its approach mid-response. This self-correction behavior emerged after applying GRPO with CoT prompting and is especially useful in educational contexts, where understanding how to recover from mistakes is as important as arriving at the correct solution.

First, let's denote the four distinct integers as a, b, c, and d. We are given that when added in pairs, the sums 10, 18, 19, 20, 21, and 29 are obtained. This gives us the following equations: a + b = 10, a + c = 18, a + d = 19, b + c = 20, b + d = 21, and c + d = 29. We can solve these equations to find the values of a, b, c, and d.

From the first equation, we can express b as b = 10 - a. Substituting this into the second equation, we get a + (10 - a) = 18, which simplifies to 10 = 18.

This is a contradiction, so we need to reconsider our approach.

Let's re-examine the equations. From the equation b + d = 21, we can express d as d = 21 - b.

Substituting this into the equation c + d = 29, we get c + (21 - b) = 29, which simplifies to c + 21 - b = 29. Rearranging the terms, we get c - b = 8. Now we have two equations: c - b = 8 and b + c = 20.

We can solve these two equations to find the values of b and c. Adding the two equations, we get 2c = 28, which simplifies to c = 14.

Substituting c = 14 into the equation b + c = 20, we get b + 14 = 20, which simplifies to b = 6.

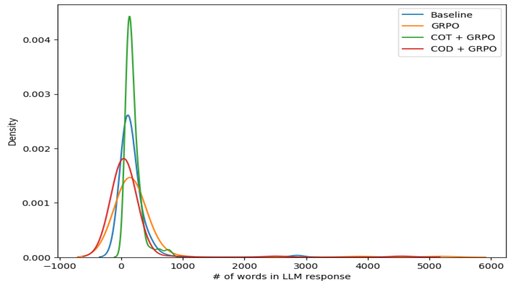
Now that we have the values of b and c, we can find the value of d. Substituting b = 6 into the equation d = 21 - b, we get d = 21 - 6 = 15.

Finally, we can find the value of a. Substituting d = 15 into the equation a + d = 19, we get a + 15 = 19, which simplifies to a = 4.

Therefore, the four integers in increasing order are 4, 6, 14, and 15.

Distribution of Words in LLM Responses

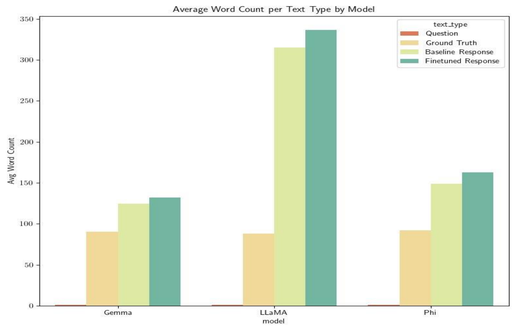
While evaluating the effectiveness of different reasoning techniques, we analyzed the distribution of word counts in LLM responses across the test set. This helped us assess how each method influenced the model's ability to generate structured, efficient, and informative answers.



The resulting density plot shows that models trained with CoT + GRPO produced responses with a more consistent word count, indicating well-structured and focused output. In contrast, the baseline model exhibited a broader spread, reflecting inconsistencies and a lack of control over verbosity. This analysis confirmed that combining structured prompting with preference-based fine-tuning encourages more reliable and concise reasoning without sacrificing depth.

Average Word Count per Text Type by Model

To understand how different models behave in response generation, we measured the average word count for various text types including questions, ground truths, and model outputs across Gemma, LLaMA, and Phi. This helped us evaluate whether fine-tuning with reasoning strategies like CoT and GRPO leads to more detailed and informative responses.



Across all model's, fine-tuned outputs are consistently longer than their baseline counterparts, underscoring the impact of step-by-step prompting in generating more complete and well-explained answers.

Evaluation Metrics

		Metrics															
	Model Name	Technique	BLEU	ROUGE 1	ROUGE 2	ROUGE L	METEOR	chrF	Factual Accuracy	Correctness (mathverify)	g-eval Logical Coherence	G-eval Score	Completeness	Step Alignment	Reasoning Length Ratio	Error Identification
	Llama 3.2 3B	GRPO	0.13450	0.20550	0.06920	0.14840	0.32690	0.32400	0.74200	0.40000	0.79040	0.35640	0.67800	0.76220	0.78010	0.77660
	Llama 3.2 3B	GRPO + COT	0.13670	0.19610	0.06780	0.14070	0.35520	0.33770	0.75450	0.55000	0.75300	0.36450	0.69820	0.79950	0.80210	0.79300
	Llama 3.2 3B	GRPO + COD	0.09630	0.13210	0.04270	0.11170	0.18430	0.21850	0.39670	0.12000	0.41400	0.28420	0.56290	0.60200	0.44210	0.52040
	Llama 3.2 1B	**DPO**	**0.08402**	**0.31323**	**0.13393**	**0.213943**	**0.32540**	**0.30130**	**0.75500**	**0.61000**	**0.88250**	**0.38378**	**0.70500**	**0.81875**	**0.30613**	**0.78000**
	Gemma3 1B	DPO	0.10519	0.37615	0.14396	0.23414	0.35420	0.33237	0.64910	0.55500	0.57000	0.34666	0.34500	0.63785	0.50956	0.52500
	Phi 3.5	DPO	0.10970	0.35512	0.13749	0.21372	0.32450	0.29986	0.68320	0.56500	0.76500	0.23135	0.50500	0.78100	0.50176	0.47500
	Llama 3.2 1B	DPO + GRPO	0.06563	0.36238	0.12508	0.27247	0.28567	0.28873	0.69180	0.59500	0.80500	0.28518	0.70500	0.67150	0.52671	0.66500
	DeepSeek R1	TOT	0.11000	0.36200	0.03500	0.07000	0.03000	0.03050	0.65130	0.44720	0.74200	0.31652	0.66800	0.78300	0.79300	0.76300
	DeepSeek R1	SC	0.00200	0.24160	0.02800	0.06000	0.03300	0.04325	0.66730	0.44570	0.12700	0.32790	0.39200	0.54300	0.51900	0.49600
	Gemma3 4B	COD	0.13530	0.31800	0.12900	0.21840	0.20230	0.27570	0.56820	0.56250	0.64700	0.35600	0.61700	0.65900	0.72100	0.71800

Summary/Conclusions

This project presents a cost-effective, transparent, and scalable framework for enhancing reasoning capabilities in small to mid-sized open-source LLMs using fine-tuning and structured prompting techniques. Through Chain-of-Thought techniques it improved step-by-step reasoning, self-correction, and educational value. The system architecture supports user accounts and preserves interaction history. The project lays a solid foundation for further innovations in explainable, accessible, and domain-specific AI tutoring systems.

Key References

[1] Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903. https://arxiv.org/abs/2201.11903

[2] Wolfe, C. (2023). *Chain-of-Thought Prompting for LLMs*. Retrieved from https://cameronrwolfe.substack.com/p/chain-of-thought-prompting-for-llms

[3] Xu, X. et al. (2023). Re-Reading Improves Reasoning in Language Models (RE2). arXiv:2309.06275. https://arxiv.org/abs/2309.06275

[4] Muennighoff, N. et al. (2025). S[1]: Simple Test-Time Scaling. arXiv:2501.19393. https://arxiv.org/abs/2501.19393

Acknowledgements

Sincere thanks to Prof. Simon Shim for his invaluable guidance and mentorship throughout the project and acknowledge the Department of Applied Data Science at San José State University for providing the academic and technical resources necessary to carry out this research.

Special thanks to open-source contributors whose tools and models made this work possible.